基础设施打造

(构建数字化办公室)

>> 今天的学习目标

基础设施打造（构建数字化办公室）

- AI编程工具

什么是Rules

Python环境部署

- QMT与miniQMT
- CASE：获取日线数据
- CASE：获取分钟数据
- CASE：获取财务数据
- 财务指标如何指导量化交易
- CASE：多因子选股

AI编程工具

Cursor编程

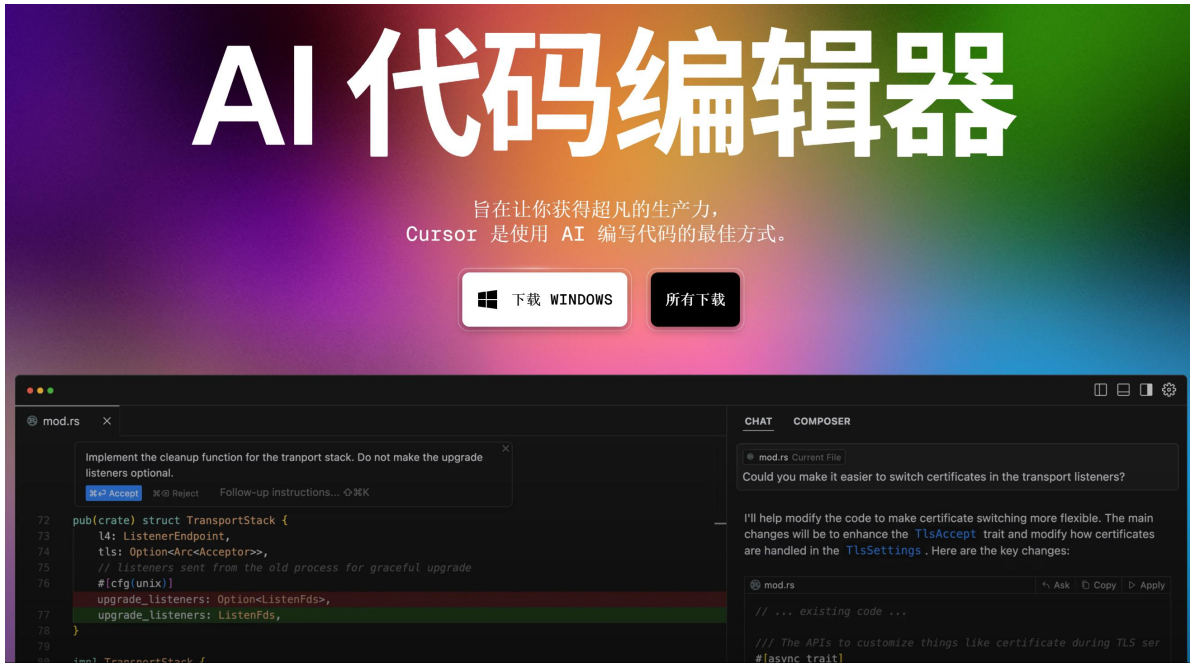
Cursor编辑器：

https://cursor.com/

Cursor 是基于 VSCode 开发的AI代码编辑器，提供智能代码补全、代码生成、代码修改、代码搜索和代码解释等。

与其他工具不同，Cursor 将 AI 辅助编码直接融入到编辑器的核心功能中，比如Cursor可以理解整个工程的代码，同时修改多个文件。

可以通过 .cursorrules 文件定制 AI 的行为。



Cursor Rules

Thinking: 什么是Cursor Rules?

Cursor Rules帮助你定制 AI 行为，让它符合你的编码风格和项目需求。

Cursor 提供两种类型的规则：

- 全局规则

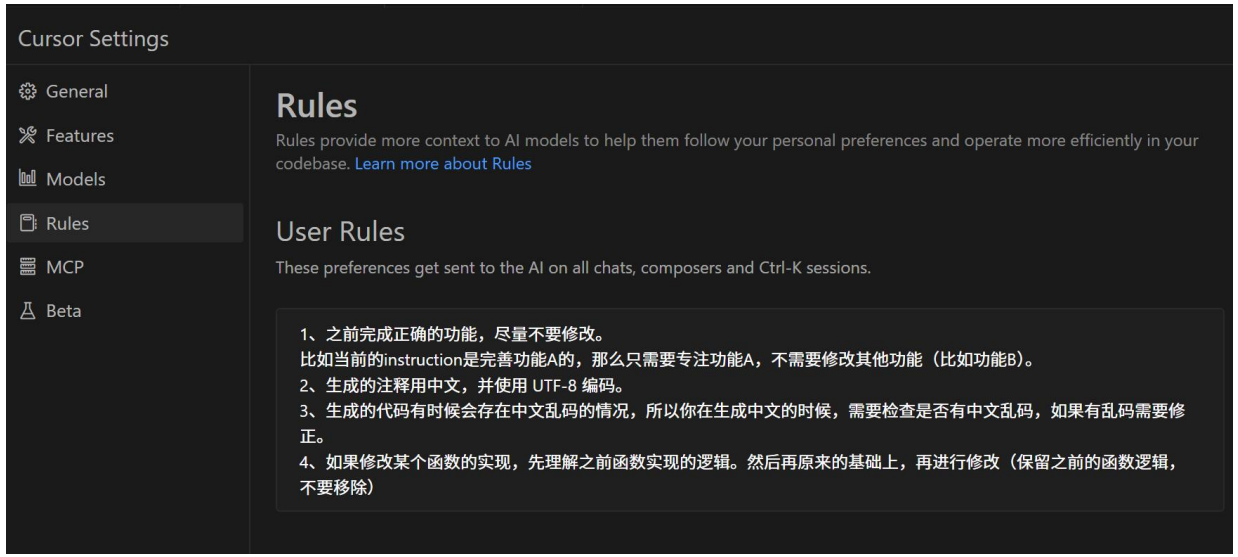
在 Cursor 设置的"通用" > "AI 规则"中配置。规则适用于你的所有项目，适合设置个人编码偏好和保持 AI 行为的一致

```
# 在 Cursor 设置 > AI 规则中

- 所有新代码使用 TypeScript
- 遵循代码整洁之道
- 优先使用 async/await 而不是回调
- 编写完善的错误处理
```

项目规则

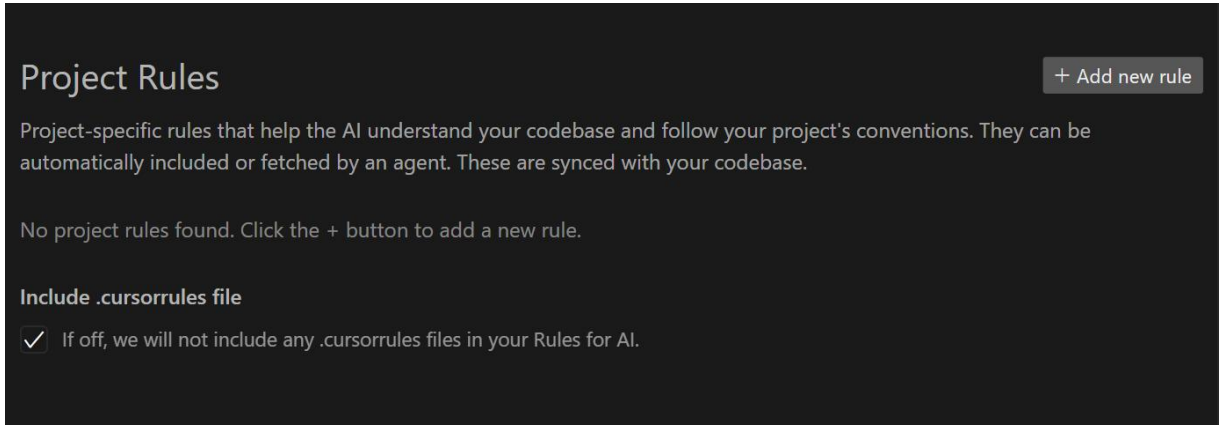
以 .mdc 格式存储在 .cursor/rules 目录中的项目特定规则。这些规则帮助 AI 理解你的代码库并遵循项目规范。通过 Cursor 设置 > 通用 > 项目规则来添加。它们会与你的代码



Cursor Rules

项目规则

仅对特定项目生效，通常存储在项目根目录的 .cursorrules 文件中。这些规则帮助 AI 理解你的代码库并遵循项目规范。通过 Cursor 设置 > 通用 > 项目规则来添加。




```
# 文件模式：*.tsx, *.ts

## React 规范
- 使用函数组件
- 实现完整的 prop 类型
- 遵循 React 最佳实践

## 样式
- 使用 Tailwind CSS
- 遵循团队样式指南

@file ../tsconfig.json
@file ../tailwind.config.js
```

项目规则优先级高于全局规则：如果项目中存在 .cursorrules 文件，则项目规则会覆盖全局规则。

全局规则：适合设置通用的编码规范、安全策略或个人偏好，确保所有项目保持一致。

项目规则：适合针对特定项目的个性化需求，例如特定技术栈、文件结构或安全要求。

Cursor编程：4种工作模式

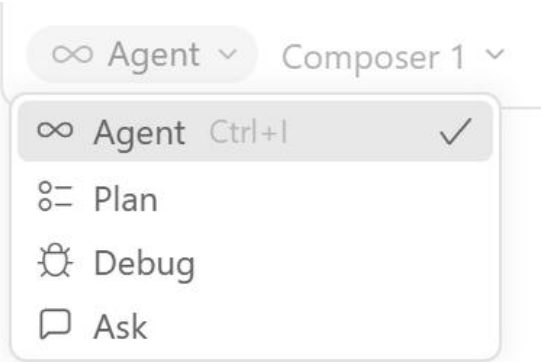
Cursor的主要功能：

- Agent（智能体）

完整的自主智能体，不仅能同时编辑多文件，更能独立执行多步骤任务：自动运行终端命令、执行测试、分析报错并迭代修复。Background Agent支持后台并行处理任务，无需等待。

- Plan（规划）

先规划后执行的模式。AI先分析需求并制定详细修改计划（含文件清单和变更说明），用户审阅确认后才执行。适合大型代码库重构或高风险修改，避免AI直接改坏代码，给你决策控制权。



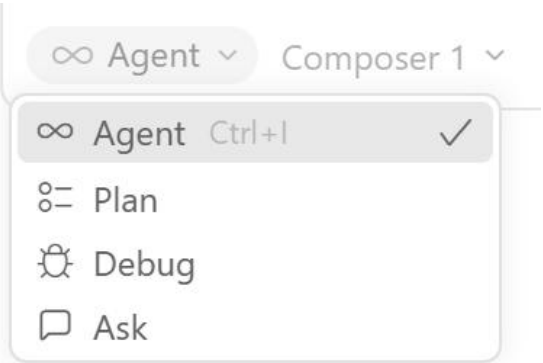
Cursor编程：4种工作模式

- Debug（调试）

专治疑难杂症的模式。粘贴报错信息后，AI自动分析堆栈、定位根因、提供修复方案，并可自动应用补丁。支持多轮追问深挖问题，适合解决编译错误、运行时异常和逻辑bug。

- Ask（询问）

纯咨询模式，不修改代码。用于解释代码逻辑、学习新技术、讨论架构方案、审查代码质量。相当于技术顾问，回答范围涵盖整个代码库，支持多轮深度技术对话，是最安全的探索性使用方式。



Cursor编程：模型选择

模式选项

- Auto：自动模式，根据任务复杂度智能选择模型，简单补调用轻量模型，复杂代码生成自动切换大模型
- MAX Mode：极限模式，解锁最大上下文窗口和最强计算能力，适合超大规模代码库分析和复杂架构设计
- Use Multiple Models：多模型协同，同时调用多模型对比结果或分工处理（如一个规划一个执行），取最优解

Cursor编程：模型选择

模型选项

- Composer 1：Cursor自研/特调的代码生成模型，专为多文件编辑和Agent任务优化，平衡质量与成本
- Opus 4.6：Anthropic旗舰大模型，最强推理能力，擅长复杂算法、架构设计和长上下文理解，适合高难度任务
- Opus 4.6 Fast：Opus的快速版本，响应更快，适合常规开发但追求速度的场景
- Sonnet 4.5：性价比之选，日常编码、调试和文档编写的最佳选择，速度和质量平衡
- GPT-5.2：OpenAI最新模型，在特定语言（如Python/JS）和框架上有优势，与Claude风格互补
- GPT-5.2 Codex：OpenAI专门优化代码的模型，擅长严格的语法逻辑、测试生成和代码审查

日常开发用 Sonnet 4.5 或 Auto；复杂重构选 Opus 4.6；需要特定OpenAI能力时切 GPT-5.2。

Python环境准备

1. Python 解释器（核心引擎）

版本要求： Python 3.10 及以上（推荐 3.11 或 3.13），以获得更好的性能和AI库支持。

https://www.python.org/downloads/

勾选 Add Python.exe to PATH => 在 Cursor 终端或命令行中直接运行 Python。

2. 核心工具库栈（依赖包）

安装完 Python 后，第三方工具箱需要通过 pip install 安装

AI 能力层： dashscope (阿里云大模型SDK)，用于调用通义千问等大模型API。

数据基础层： Pandas (金融数据处理标准)、Tushare (历史行情下载)。

实盘交易层： xtquant (QMT专用Python库)，连接券商客户端进行下单与实时行情接收。

Python环境准备

3. 交互式分析工具：Jupyter（可选）

量化交易不仅是写代码，更是数据探索。

使用Jupyter Notebook 可以交互式的运行，了解每块的代码执行结果

优势：它允许你写一行代码，看一眼结果，适合调试策略逻辑、绘制K线图以及即时查看中间数据变量。

4. AI编程工具 （必备）

国内：Trae, Lingma, Code Buddy, wucai code

国外：Cursor, Claude Code

QMT 与 miniQMT

QMT软件与 xtquant

QMT（Quantitative Trading Platform）：

- 是迅投推出的本地量化终端，集行情、回测、实盘、风控于一体
- 支持 Python/VBA 双语言，全内存下单延迟 < 1 ms，
- 可交易股票、期货、期权、ETF 等全品种。

XtQuant：

- 官方开源的 Python 接口包，把 QMT 的交易与行情能力封装成简洁 API，
- 用户可在 PyCharm/Jupyter 中调用下单、查持仓、订阅行情，实现“策略与终端分离”，轻松对接自己的回测框架或 AI 模型
- 占用资源更低，适合程序化交易者

QMT软件

Step 1： 下载券商QMT

http://www.ebscn.com/software/

官网--软件下载--专业软件

金阳光QMT软件

35D8AF5773F78E03A2A745A4D231B2BF 更新时间：2025-11-08

软件介绍：金阳光QMT是基于沪深交易所PC端Level-2行情的行情交易客户端，
能，包括沪深十档行情、深圳千档行情、沪深逐笔成交队列、深圳委托队列、星
算法交易、组合交易、风控管理等更多样化和可控的交易方式。详情

软件下载 用户手册

QMT软件

Step 2，登录QMT软件

用户名为8位资金账号，初始密码同用户名

登录后，请及时修改初始密码。

Step 3，绑定资金账户

可以参考在线文档

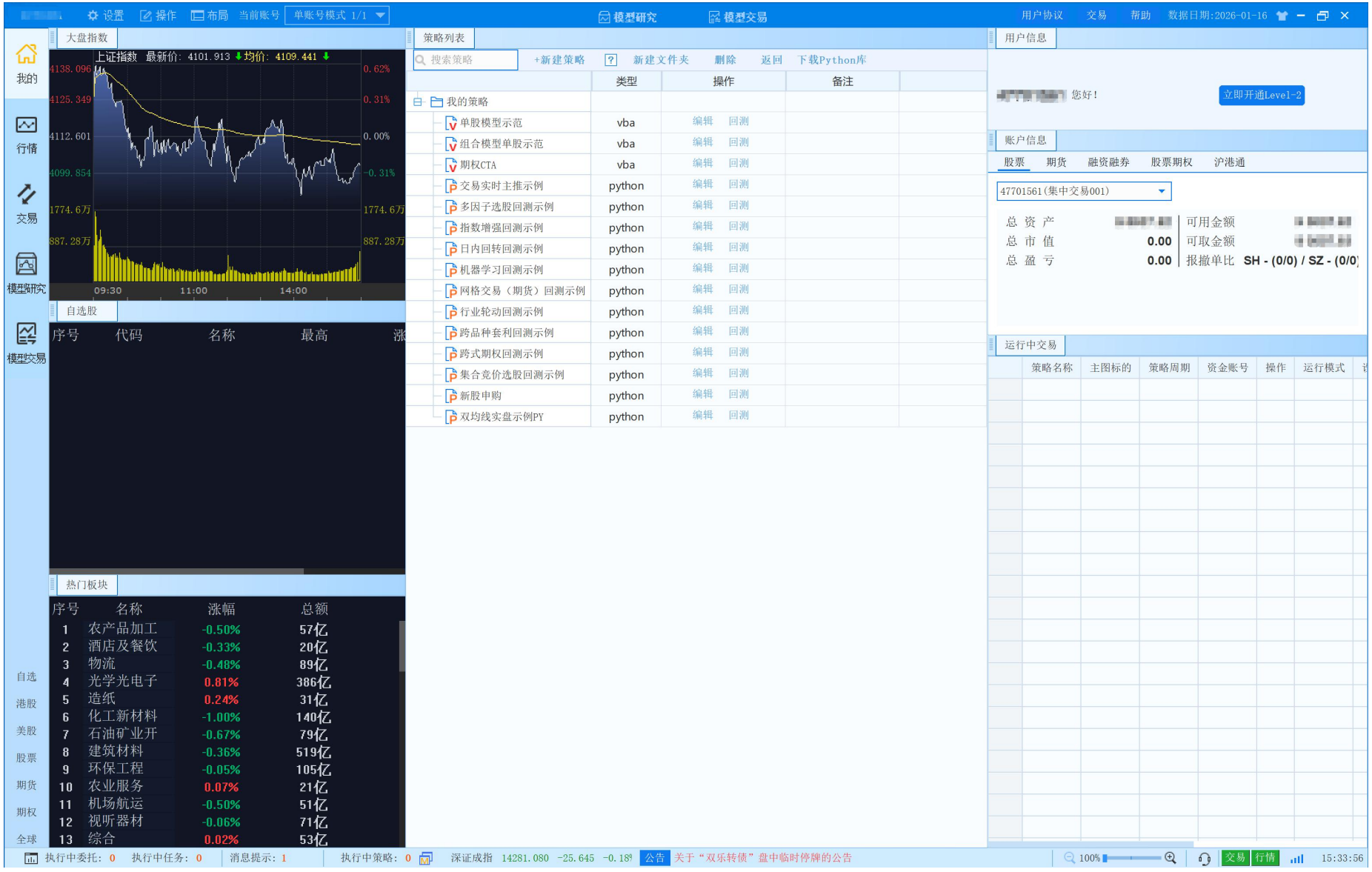
http://www.ebscn.com/plat_files/upload/png_upload/20250415/20250415174470328755 2.pdf

QMT软件

选中独立交易，即进入到 miniQMT模式

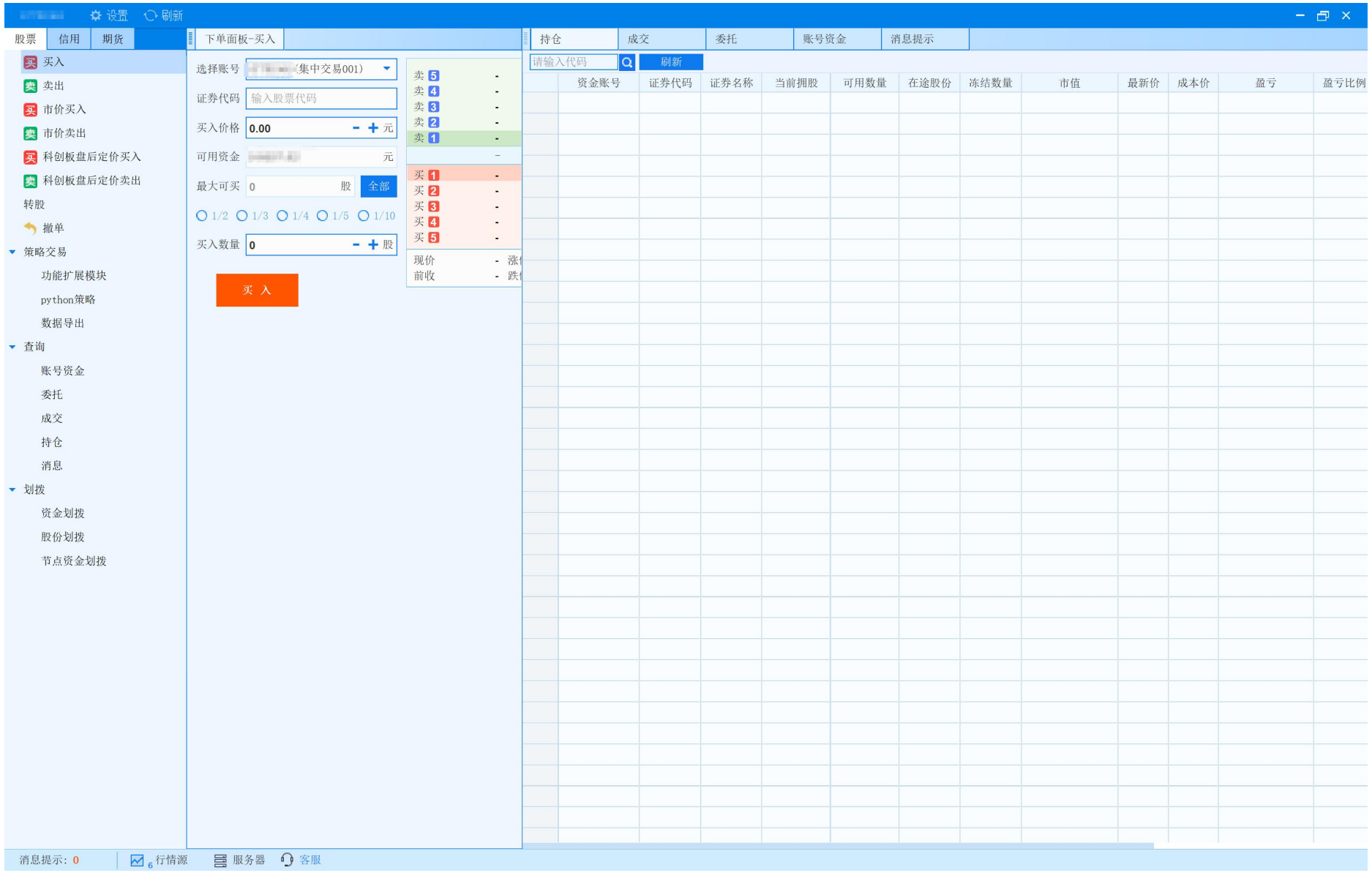
没有选中独立交易，即是 QMT模式

QMT软件


设置
操作
布局
当前账号
单账号模式 1/1
模型研究
模型交易
用户协议
交易
帮助
数据日期:2026-01-16
我的
行情
交易
模型研究
模型交易
大盘指数
上证指数 最新价: 4101.913 均价: 4109.441
4138.096
4125.349
4112.601
4099.854
1774.6万
887.28万
0.62%
0.31%
0.00%
-0.31%
09:30
11:00
14:00
自选股
序号 代码 名称 最高
热门板块
序号 名称 涨幅 总额
1 农产品加工 -0.50% 57亿
2 酒店及餐饮 -0.33% 20亿
3 物流 -0.48% 89亿
4 光学光电子 0.81% 386亿
5 造纸 0.24% 31亿
6 化工新材料 -1.00% 140亿
7 石油矿业开 -0.67% 79亿
8 建筑材料 -0.36% 519亿
9 环保工程 -0.05% 105亿
10 农业服务 0.07% 21亿
11 机场航运 -0.50% 51亿
12 视听器材 -0.06% 71亿
13 综合 0.02% 53亿
自选
港股
美股
股票
期货
期权
全球
策略列表
搜索策略
+新建策略
新建文件夹
删除
返回
下载Python库
类型
操作
备注
我的策略
单股模型示范 vba 编辑 回测
组合模型单股示范 vba 编辑 回测
期权CTA vba 编辑 回测
交易实时主推示例 python 编辑 回测
多因子选股回测示例 python 编辑 回测
指数增强回测示例 python 编辑 回测
日内回转回测示例 python 编辑 回测
机器学习回测示例 python 编辑 回测
网格交易（期货）回测示例 python 编辑 回测
行业轮动回测示例 python 编辑 回测
跨品种套利回测示例 python 编辑 回测
跨式期权回测示例 python 编辑 回测
集合竞价选股回测示例 python 编辑 回测
新股申购 python 编辑 回测
双均线实盘示例PY python 编辑 回测
用户信息
您好！
立即开通Level-2
账户信息
股票
期货
融资融券
股票期权
沪港通
47701561(集中交易001)
总 资 产
总 市 值 0.00
总 盈 亏 0.00
可用金额
可取金额
报撤单比 SH - (0/0) / SZ - (0/0)
运行中交易
策略名称
主图标的
策略周期
资金账号
操作
运行模式
执行中委托: 0
执行中任务: 0
消息提示: 1
执行中策略: 0
深证成指 14281.080 -25.645 -0.18%
公告
关于“双乐转债”盘中临时停牌的公告
100%
交易
行情
15:33:56

QMT模式

miniQMT软件



miniQMT模式

QMT与miniQMT的区别

Thinking：QMT与miniQMT的区别是什么？

- 功能完整度

QMT：集行情、研究、回测、实盘、风控于一体的全栈平台，支持 Python/VBA 开发，可本地/云端运行

miniQMT：只保留“实盘交易 + 行情接口”的极简子系统，不提供回测、图形界面和高级风控模块 。

- 策略开发方式

QMT：策略必须写在 QMT 客户端内部，用官方编辑器或内置 Python 环境运行。

miniQMT：以 XtQuant Python 包形式独立安装，策略可在任意 IDE（PyCharm、Jupyter 等）里编写，通过 API 把指令发回券商柜台下单，实现“策略与交易分离” 。

QMT与miniQMT的区别

- 运行环境

QMT：需要一直开着 QMT 客户端，策略才能跑。

miniQMT：只需后台运行一个轻量化代理进程，甚至可以放到云服务器，占用资源更少，延迟更低（官方称订单延迟可低至 5 ms）。

- 适用人群

QMT：适合需要图形回测、多品种多账户管理、对风控要求高的私募或高净值个人。

miniQMT：适合程序猿/量化爱好者，只想用 Python 自由写策略、对回测和界面无刚需、追求轻量和低延迟

QMT 是大而全的量化工作站；miniQMT 是把交易接口抽出来的小而快的 Python 量化利器。

QMT申请

Step1，网页链接信息填写

（1）个人信息

注意不要修改营业部相关信息，只填自己相关信息即可

身份证照片背景需一致，身份证露出四个边

家庭住址需精确到门牌号

拍摄视频时需要背景没有人经过，无杂音

（2）账户选择

开通账户按照默认的即可

点击【下一步】后，需要前往手机APP市场下载券商APP，点击我要开户并填入和此前网页注册里一样的号码（此时会自动填入之前填入的信息）

class.wucai.com 网站中有banner，券商宣传链接

QMT申请

Step2，APP信息填写

进入券商APP后，继续填写信息并提交开户。

Step3，接听回访电话

Step4，开户成功

开户成功后，如果期间收到短信指出开户信息有误，重新登入APP再次对应修改即可；如果APP没更新，关掉APP重新打开则会自动更新。

Step5，保存账号信息

保存接收到的证券资金账户等账号信息（个人私密信息、请勿外传或在群内分享），并在APP中对应录入。

QMT申请

- 通过渠道，可享受较低的资金门槛。

不要直接在APP市场下载后自行开户。

- 部分银行需转账进行激活

如果遇到像招商银行、浦发银行、平安银行在首次开户需要进行银证转账进行激活，可转1元打通即可，以后就可以直接在券商APP进行转账了。

- 服务经理

开户完成后，系统会自动分配服务经理。服务经理会协助你开通 QMT 或 miniQMT 的使用权限，后续的使用问题也可以找服务经理协助。

QMT申请

QMT开户和权限开通流程按工作日计算，以下是理想情况下的时间节点：

第一天：开户成功，并出股东账号

第二天：分配服务经理，可在【我的-帮助中心】中查看

第三天：可以在APP上可以看到服务经理，并添加企微联系

同时可以发起程序化交易报备

第五天：程序化交易报备完成

同时可以发起QMT终端申请

第六天：QMT终端申请完成，可以登录QMT

以上流程是比较极致的情况，中间可能会遇到一些材料不全的问题，服务经理会积极联系大家进行补齐。

QMT常见问题

1. QMT开通后，自带的免费的，无需额外申请的数据包含哪些呢？

A股的行情数据和股票等基本数据都能获取，level 2的功能和数据才需要单独收费

2. 如果想要开通港股、期货、期权等其他数据，这个是找客户经理额外申请吗？费用大概多少呢？

支持港股通和期权、期货需要看具体是哪家期货公司、港股不支持——这些开通只收取佣金（具体佣金和服务经理谈）、不收取开通费用

3. 如果人在国外，国外的手机号是否可以申请开通 QMT？

需要国内手机号，找个国内的手机号

4. 如果人在国外，可以使用 QMT 来实盘交易或者模拟盘交易吗？QMT 软件在使用过程中是否不允许使用翻墙软件？

国外无法登陆QMT，需要使用VPN

CASE：获取日线数据

CASE： 获取日线数据

TO DO：以 贵州茅台（600519.SH）为例，分别使用 QMT（miniQMT）、Tushare Pro、AkShare 三个数据源，下载该股票的历史日线行情数据，保存为 CSV 文件

日期范围： 20240101 ~ 20251231

复权方式：前复权（qfq）

字段	说明
date	交易日期
open	开盘价
high	最高价
low	最低价
close	收盘价
volume	成交量

CASE：获取日线数据（QMT）

安装 QMT 客户端，配置好 xtquant Python 包

运行前必须启动 miniQMT 客户端并登录

QMT 获取日线数据分两步：先下载到本地缓存，再从缓存中读取

```
# 1. 下载历史数据到本地缓存
xtdata.download_history_data(
    stock_code='600519.SH',
    period='1d',
    start_time='20240101'
)
```

```
# 2. 从缓存中获取数据（返回字典，每个字段是一个 DataFrame）
res = xtdata.get_market_data(
    stock_list=['600519.SH'],
    period='1d',
    start_time='20240101',
    end_time='',
    count=-1,
    dividend_type='front'，# 前复权
    fill_data=True
)
```

日线数据-QMT.py

CASE：获取日线数据（QMT）

```
# 3. 提取各字段（res 是 dict，key 为字段名，value 为 DataFrame）
#    DataFrame 的 index 是股票代码，columns 是日期
close_prices = res['close'].loc['600519.SH']
open_prices  = res['open'].loc['600519.SH']
# ... 类似提取 high, low, volume
```

开始下载股票数据

股票：贵州茅台(600519.SH)

日期范围：20240101 至 20251231

步骤1：下载历史数据...

* xtdata连接成功 2026-02-10 19:49:12*

服务信息: {'tag': 'sp3', 'version': '1.0'}

服务地址: 127.0.0.1:58610

数据路径: D:\光大证券金阳光QMT实盘\bin.x64/../userdata_mini/datadir

设置xtdata.enable_hello = False可隐藏此消息

下载完成，等待数据写入...

步骤2：获取历史收盘价数据...

成功获取 508 条历史数据

数据日期范围：2024-01-02 至 2026-02-10

步骤3：保存数据到CSV文件...

数据已保存至：D:\推荐系统\量化交易-2\3-打造数字化办公室\CASE-数采集\data\600519_SH_daily_QMT.csv

数据预览（前5行）：

date	close	open	high	low	volume
2024-01-02	1578.622	1608.612	1611.802	1571.712	32156
2024-01-03	1587.612	1574.722	1588.832	1569.942	20229
2024-01-04	1562.612	1586.612	1586.612	1556.542	21551
2024-01-05	1556.972	1554.942	1572.272	1545.722	20243
2024-01-08	1537.602	1554.612	1555.612	1533.622	25586

日线数据-QMT.py

CASE：获取日线数据（QMT）

QMT技术点：

- 数据结构：get_market_data() 返回的是一个嵌套字典，形如 {'close': DataFrame, 'open': DataFrame, ...}，其中每个 DataFrame 的行是股票代码，列是日期（YYYYMMDD 格式）
- 日期格式转换：QMT 返回的日期列名是 YYYYMMDD 字符串，需用 pd.to_datetime(format='%Y%m%d') 转换
- 下载等待：download_history_data() 返回后，需要 time.sleep(2) 等待数据写入本地缓存

CASE：获取日线数据（tushare）

设置环境变量 TUSHARE_TOKEN（需要 Tushare Pro 积分 >= 120）

Tushare 提供 ts.pro_bar() 接口，一次调用直接返回带复权的日线数据：

pip install tushare

```
# 1. 初始化
ts.set_token(os.environ.get("TUSHARE_TOKEN"))
pro = ts.pro_api()

# 2. 一次调用获取前复权日线数据
df = ts.pro_bar(
    ts_code='600519.SH',
    start_date='20240101',
    end_date='20251231',
    adj='qfq',   # 前复权（qfq=前复权, hfq=后复权, None=不复权）
    freq='D'     # 日线
)

# 3. 整理列名（Tushare 的日期列叫 trade_date，成交量列叫 vol）
df = df.rename(columns={'trade_date': 'date', 'vol': 'volume'})
```

日线数据-tushare.py

CASE：获取日线数据（tushare）

tushare技术点：

- 列名映射：Tushare 返回的列名和通用命名不同，需要 rename：trade_date -> date，vol -> volume
- 日期格式：返回的 trade_date 是 YYYYMMDD 字符串，需转换为 datetime
- 排序方向：Tushare 默认按日期降序返回，需要 sort_values('date') 转为升序

CASE：获取日线数据（akshare）

无需 Token，完全免费

AkShare 提供 stock_zh_a_hist() 接口，数据来源于东方财富

pip install akshare

```
# 1. 一次调用获取前复权日线数据
df = ak.stock_zh_a_hist(
    symbol='600519',          # 纯数字，不带交易所后缀
    period='daily',           # 日线（可选：daily/weekly/monthly）
    start_date='20240101',
    end_date='20251231',
    adjust='qfq'              # 前复权
)
# 2. 整理列名（AkShare 返回中文列名）
col_map = {
    '日期': 'date',
    '开盘': 'open',
    '收盘': 'close',
    '最高': 'high',
    '最低': 'low',
    '成交量': 'volume',
}
df = df.rename(columns=col_map)
```

日线数据-

CASE：获取日线数据（akshare）

Akshare技术点：

- 股票代码格式：AkShare 使用纯 6 位数字（600519），不带 .SH 后缀
- 中文列名：返回的列名是中文（日期、开盘、收盘...），需要做一次重命名映射
- 数据来源：来自东方财富，数据质量和实时性较好，且完全免费无限制

Summary

对比项	QMT（miniQMT）	Tushare Pro	AkShare
费用	QMT 客户端（券商免费提供）	需 120+ 积分	免费
需要 Token	否（需启动客户端）	是（TUSHARE_TOKEN）	否
数据来源	QMT 本地数据	Tushare 服务器	东方财富
核心接口	xtdata.get_market_data()	ts.pro_bar()	ak.stock_zh_a_hist()
代码复杂度	低（两步：下载+获取）	低（一步）	低（一步）
列名格式	英文	英文（需 rename）	中文（需 rename）
复权支持	dividend_type='front'	adj='qfq'	adjust='qfq'

CASE：获取分钟数据

CASE：获取分钟数据

TO DO：以 贵州茅台（600519.SH）为例，分别使用 QMT（miniQMT）、Tushare Pro、AkShare 三个数据源，下载该股票 2026-02-10 当天的 1 分钟 K 线数据

分钟数据和日线数据的获取逻辑几乎一样，只是 period / freq 参数从日线改为分钟。

主要区别在于 数据权限：

- QMT：免费，无特殊限制
- Tushare：分钟数据需要单独捐助 1000 元开通权限，普通积分用户无法获取
- AkShare：免费，但只能获取最近 5 个交易日的分钟数据

CASE：获取分钟数据（QMT）

Thinking：一个交易日一共有多少根 分钟K线？

上午：09:30 ~ 11:30 = 120 根

下午：13:00 ~ 15:00 = 121 根（含 15:00 收盘那根）

A 股一个交易日共 241 根 1 分钟 K 线

```
# 1. 连接 miniQMT
xtdata.connect()

# 2. 下载分钟数据到本地缓存
xtdata.download_history_data(
    stock_code='600519.SH',
    period='1m',            # 1分钟K线（可选：1m, 5m, 15m, 30m, 60m）
    start_time='20260210'
```

```
# 3. 从缓存中获取数据
res = xtdata.get_market_data(
    stock_list=['600519.SH'],
    period='1m',
    start_time='20260210',
    end_time='20260211',     # 设为下一天，确保包含全天数据
    count=-1,
    dividend_type='front',
    fill_data=False
)
# 4. 提取字段（和日线完全一样）
close_prices = res['close'].loc['600519.SH']
```

分钟数据-QMT.py

CASE：获取分钟数据（tushare）

和日线数据一样使用 ts.pro_bar()，只是 freq 从 'D' 改为 '1min

```
# 1. 连接 miniQMT
xtdata.connect()

# 2. 下载分钟数据到本地缓存
xtdata.download_history_data(
    stock_code='600519.SH',
    period='1m',            # 1分钟K线（可选：1m, 5m, 15m, 30m, 60m）
    start_time='20260210'
```

```
# 3. 从缓存中获取数据
res = xtdata.get_market_data(
    stock_list=['600519.SH'],
    period='1m',
    start_time='20260210',
    end_time='20260211',     # 设为下一天，确保包含全天数据
    count=-1,
    dividend_type='front',
    fill_data=False
)
# 4. 提取字段（和日线完全一样）
close_prices = res['close'].loc['600519.SH']
```

分钟数据-QMT.py

CASE：获取分钟数据（tushare）

和日线数据一样使用 ts.pro_bar()， 只是 freq 从 'D' 改为 '1min'：

```
# 1. 初始化
ts.set_token(os.environ.get("TUSHARE_TOKEN"))
pro = ts.pro_api()

# 2. 获取1分钟数据（前复权）
df = ts.pro_bar(
    ts_code='600519.SH',
    start_date='20260210',
    end_date='20260210',
    adj='qfq',
    freq='1min'            # 1分钟（可选：1min, 5min, 15min,
30min, 60min）
)
```

```
# 3. 整理列名
df = df.rename(columns={
    'trade_time': 'datetime',  # 分钟数据返回 trade_time
  （日线返回 trade_date）
    'vol': 'volume'
})
```

分钟数据-tushare.py

CASE：获取分钟数据（akshare）

限制：只能获取最近 5 个交易日的分钟数据

AkShare 提供 stock_zh_a_hist_min_em() 接口，数据来源于东方财富：

```
# 1. 一次调用获取分钟数据
df = ak.stock_zh_a_hist_min_em(
    symbol='600519',                    # 纯数字，不带后缀
    start_date='2026-02-10 09:30:00',   # 注意：日期格式含时分秒
    end_date='2026-02-10 15:00:00',
    period='1',                         # 1分钟（可选：1, 5, 15, 30, 60）
    adjust=''                           # 不复权
)
```

```
# 2. 整理列名（中文 -> 英文）
col_map = {
    '时间': 'datetime',
    '开盘': 'open',
    '收盘': 'close',
    '最高': 'high',
    '最低': 'low',
    '成交量': 'volume',
    '成交额': 'amount',
}
df = df.rename(columns=col_map) 分钟数据-
```

Summary

对比项	QMT（miniQMT）	Tushare Pro	AkShare
费用	QMT 客户端（券商免费）	捐助 1000 元	免费
需要 Token	否（需启动客户端）	是	否
数据来源	QMT 本地数据	Tushare 服务器	东方财富
核心接口	get_market_data(period='1m')	pro_bar(freq='1min')	stock_zh_a_hist_min_em(period='1')

推荐 QMT：如果已有券商 QMT 客户端，分钟数据获取最方便，无付费门槛、无时间限制

AkShare 适合临时用：只能拿最近 5 天的数据，适合盘中监控或当天回测

CASE：获取财务数据

CASE：获取财务数据

TO DO：以 贵州茅台（600519.SH）为例，分别使用 QMT（miniQMT）、Tushare Pro、AkShare 三个数据源，下载该股票的财务数据，保存为 CSV 文件

指标范围：尽量覆盖每股指标、盈利能力、偿债能力、营运能力、现金流、规模等

- Tushare:

直接返回计算好的指标，使用字段名即可

- Akshare：

返回原始三大报表，需要提取 + 计算

- QMT：

返回原始五张报表，需要提取 + 计算

CASE：获取财务数据

类别	指标	字段名
每股指标	基本每股收益	eps
	每股净资产	bps
	每股经营现金流	ocfps
盈利能力	净资产收益率 ROE	roe
	总资产报酬率 ROA	roa
	销售毛利率	grossprofit_margin
	销售净利率	netprofit_margin
	营业利润率	op_margin
偿债能力	资产负债率	debt_to_assets
	流动比率	current_ratio
	速动比率	quick_ratio

类别	指标	字段名
营运能力	总资产周转率	assets_turn
现金流	经营活动现金流净额	operating_cashflow
	投资活动现金流净额	investing_cashflow
	筹资活动现金流净额	financing_cashflow
	经营现金流/营业收入	ocf_to_revenue
	经营现金流/净利润	ocf_to_profit
规模	营业收入	revenue
	净利润	net_profit
	总资产	total_assets
	总负债	total_liab
	所有者权益	total_equity

CASE：获取财务数据（tushare）

Tushare 提供 fina_indicator 接口，一次调用直接返回计算好的财务指标，不需要自己从报表里算：

```
# 1. 定义要获取的45个字段
FINA_FIELDS = ",".join([
    "ts_code", "ann_date", "end_date",
    "eps", "bps", "roe", "roa",            # 每股 + 盈利
    "grossprofit_margin", "netprofit_margin",  # 利润率
    "debt_to_assets", "current_ratio",       # 偿债
    "netprofit_yoy", "or_yoy",               # 成长
    "assets_turn", "inv_turn",               # 营运
    "fcff", "ocf_to_or",                     # 现金流
    # ... 共45个字段
])
```

```
# 2. 一次API调用，拿到所有报告期的指标
df = pro.fina_indicator(ts_code='600519.SH',
fields=FINA_FIELDS)

# 3. 排序 + 保存
df = df.sort_values('end_date')
df.to_csv('data/600519_SH_fina_tushare.csv')
```

财务数据-QMT.py

CASE：获取财务数据（tushare）

Tushare获取的财务字段：

- 每股指标（6个）: eps, dt_eps, bps, ocfps, undist_profit_ps, total_revenue_ps
- 盈利能力（9个）: roe, roe_waa, roe_dt, roa, grossprofit_margin, netprofit_margin, profit_to_gr, op_of_gr, ebit_of_gr
- 偿债能力（4个）: debt_to_assets, current_ratio, quick_ratio, cash_ratio
- 成长能力（8个）: netprofit_yoy, dt_netprofit_yoy, or_yoy, op_yoy, ocf_yoy, bps_yoy, assets_yoy, eqt_yoy
- 营运能力（7个）: assets_turn, inv_turn, ar_turn, ca_turn, fa_turn, invturn_days, arturn_days
- 现金流/收益质量（8个）: fcff, fcfe, salescash_to_or, ocf_to_or, ocf_to_opincome, op_income, ebit, ebitda

财务数据-QMT.py

CASE：获取财务数据（akshare）

AkShare 没有财务指标汇总接口，需要自己从三大报表里提取原始数据再计算指标

```
# 1. 下载三大报表（每次返回该股票全部历史，约100期）
df_income   = ak.stock_financial_report_sina(stock='sh600519', symbol='利润表')
df_balance  = ak.stock_financial_report_sina(stock='sh600519', symbol='资产负债表')
df_cashflow = ak.stock_financial_report_sina(stock='sh600519', symbol='现金流量表')

# 2. 将三张报表按报告期对齐（用第一列做日期索引）
#    注意：不能用列名匹配日期列（编码可能有差异），要用位置索引
df['_date'] = df.iloc[:, 0].astype(str).str.replace('-', '').str[:8]
```

财务数据-QMT.py

CASE：获取财务数据（akshare）

```
# 3. 按报告期遍历，从每张报表中提取字段
for period in all_periods:
    inc = income_map[period]    # 利润表该期数据
    bal = balance_map[period]   # 资产负债表该期数据
    cf  = cashflow_map[period]  # 现金流量表该期数据

    # 提取原始值（列名是中文，用模糊匹配）
    revenue      = get_col(inc, ['营业收入', '一、营业收入'])
    net_profit   = get_col(inc, ['净利润', '五、净利润'])
    total_assets = get_col(bal, ['资产总计'])
    total_equity = get_col(bal, ['所有者权益合计'])
    operating_cf = get_col(cf,  ['经营活动产生的现金流量净额
'])
```

```
# 手动计算派生指标
roe = net_profit / total_equity * 100
roa = net_profit / total_assets * 100
grossprofit_margin = (revenue - cost) / revenue * 100
debt_to_assets = total_liab / total_assets * 100
current_ratio = current_assets / current_liab
```

财务数据-akshare.py

CASE：获取财务数据（akshare）

akshare获取的财务字段（22个）：

- 每股指标（1个）： eps(基本每股收益)
- 盈利能力（5个）： roe(净资产收益率, 手动计算), roa(总资产报酬率, 手动计算), grossprofit_margin(销售毛利率, 手动计算), netprofit_margin(销售净利率, 手动计算), op_margin(营业利润率, 手动计算)
- 偿债能力（3个）： debt_to_assets(资产负债率, 手动计算), current_ratio(流动比率, 手动计算), quick_ratio(速动比率, 手动计算)
- 成长能力（0个）：无（需跨期对比计算，原始报表不直接提供）
- 营运能力（1个）： assets_turn(总资产周转率, 手动计算)

CASE：获取财务数据（akshare）

- 现金流/收益质量（5个）：operating_cashflow(经营活动现金流净额), investing_cashflow(投资活动现金流净额), financing_cashflow(筹资活动现金流净额), ocf_to_revenue(经营现金流/营收, 手动计算), ocf_to_profit(经营现金流/净利润, 手动计算)
- 规模指标（6个）：revenue(营业收入), net_profit(净利润), total_assets(资产总计), total_liab(负债合计), total_equity(所有者权益合计), monetary_funds(货币资金)

没有 bps(每股净资产)，因为新浪报表不提供总股本，无法计算。没有成长能力和详细营运周转率指标。但有规模指标的绝对值数据，这是 Tushare 没有的

CASE：获取财务数据（QMT）

和 AkShare 类似，QMT 也是返回原始报表，需要自己提取和计算。但 QMT 有五张报表

```
TABLE_LIST = ['Balance', 'Income', 'CashFlow', 'PershareIndex', 'Capital']
# 1. 连接 miniQMT
xtdata.connect()

# 2. 异步下载（必须用 download_financial_data2，同步版会卡死）
for table_name in TABLE_LIST:
    xtdata.download_financial_data2(
        stock_list=['600519.SH'],
        table_list=[table_name],
        callback=on_download_done     # 通过回调通知完成
    )
# 等待所有回调完成...
```

财务数据-QMT.py

CASE：获取财务数据（QMT）

```
# 3. 从本地缓存获取数据
data = xtdata.get_financial_data(
    stock_list=['600519.SH'],
    table_list=TABLE_LIST,
    start_time='20150101',
    end_time='20261231',
    report_type='report_time'
)

# 4. 将五张报表按报告期对齐
pershare_map =
build_period_map(data['600519.SH']['PershareIndex'])
balance_map  =
build_period_map(data['600519.SH']['Balance'])
income_map   =
```

```
# 5. 遍历报告期，从各表中提取字段
for period in all_periods:
    ps  = pershare_map[period]   # 每股指标
    bal = balance_map[period]    # 资产负债表
    inc = income_map[period]     # 利润表
    cf  = cashflow_map[period]   # 现金流量表

    eps = ps['s_fa_eps_basic']
    bps = ps['s_fa_bps']
    roe = ps['du_return_on_equity']
    revenue = inc['revenue']
    total_assets = bal['tot_assets']
    operating_cf = cf['net_cash_flows_oper_act']
    # ...手动计算 ROA、毛利率、负债率等
```

财务数据-QMT.py

CASE：获取财务数据（QMT）

QMT注意细节：

坑1：download_financial_data() 会无限卡住

download_financial_data() 同步阻塞，会卡死，不能用

download_financial_data2() 异步回调，用这个。通过 callback 接收完成通知，循环等待。

```
done_count = [0]
def on_download_done(data):
    done_count[0] += 1
for table_name in TABLE_LIST:
    xtdata.download_financial_data2(
        stock_list=[STOCK_CODE],
        table_list=[table_name],
        callback=on_download_done
    )
```

```
# 等待所有表下载完成
for _ in range(60):
    if done_count[0] >= len(TABLE_LIST):
        break
    time.sleep(1)
```

财务数据-QMT.py

CASE：获取财务数据（QMT）

坑2：必须先 xtdata.connect()

在调用任何 download_* 或 get_* 函数之前，必须先建立与 miniQMT 客户端的连接

```
xtdata.connect()  # 如果 miniQMT 没启动，这里会报错
```

坑3：m_timetag 是字符串不是时间戳

文档上说 m_timetag 是时间戳，但实际返回的是日期字符串 "20200331"。代码必须兼容两种格式

```
def normalize_timetag(ts_val):
    s = str(ts_val).strip()
    if len(s) == 8 and s.isdigit():
        return s            # 已经是 "20200331" 格式
    # 否则当作时间戳处理...
```

财务数据-QMT.py

CASE：获取财务数据（QMT）

坑4：字段名和文档不一致

xtquant 的字段名和你直觉想的不同，需要运行一次打印出来看

我们期待的字段名	实际字段名	所在报表
eps	s_fa_eps_basic	PershareIndex
bps	s_fa_bps	PershareIndex
roe	du_return_on_equity	PershareIndex
revenue	revenue	Income
total_assets	tot_assets	Balance
total_liabilities	tot_liab	Balance
total_equity	total_equity	Balance
operating_cashflow	net_cash_flows_oper_act	CashFlow
investing_cashflow	net_cash_flows_inv_act	CashFlow
financing_cashflow	net_cash_flows_fnc_act	CashFlow
gross_profit_margin	sales_gross_profit	PershareIndex

财务数据-QMT.py

CASE：获取财务数据（QMT）

输出字段（26个）

- 每股指标（4个）： eps(基本每股收益, PershareIndex), bps(每股净资产, PershareIndex), ocfps(每股经营现金流, PershareIndex), undist_profit_ps(每股未分配利润, PershareIndex)
- 盈利能力 （5个） roe(净资产收益率, PershareIndex), roa(总资产报酬率, 手动计算), grossprofit_margin(销售毛利率, PershareIndex), netprofit_margin(销售净利率, 手动计算), op_margin(营业利润率, 手动计算)
- 偿债能力 （3个）debt_to_assets(资产负债率, 手动计算), current_ratio(流动比率, 手动计算), quick_ratio(速动比率, 手动计算)
- 成长能力： 无（QMT 原始报表不提供同比数据）
- 营运能力（1个）assets_turn(总资产周转率, 手动计算)

财务数据-QMT.py

CASE：获取财务数据（QMT）

- 现金流/收益质量（5个）：operating_cashflow(经营活动现金流净额, CashFlow), investing_cashflow(投资活动现金流净额, CashFlow), financing_cashflow(筹资活动现金流净额, CashFlow), ocf_to_revenue(经营现金流/营收, 手动计算), ocf_to_profit(经营现金流/净利润, 手动计算)

- 规模指标（7个）：total_assets(资产总计, Balance), total_liab(负债合计, Balance), total_equity(所有者权益合计, Balance), revenue(营业收入, Income), net_profit(净利润, Income), monetary_funds(货币资金, Balance), total_shares(总股本, Capital)

相比 AkShare 多了 bps、ocfps、undist_profit_ps、total_shares 四个字段（得益于 PershareIndex 和 Capital 两张额外报表）。但同样缺少成长能力和详细营运周转率指标。

财务数据-QMT.py

Summary

对比项	Tushare Pro	AkShare	QMT（miniQMT）
费用	需 2000+ 积分	免费	QMT 客户端（券商提供）
需要 Token	是	否	否（需启动客户端）
数据来源	Tushare 服务器	新浪财经	QMT 本地数据
数据形式	预计算的指标	原始三大报表	原始五张报表

打卡：获取你的自选股数据

针对你的自选股，获取以下的数据：

- 日线，比如 20240101 至 20251231
- 分钟线，比如 20260206 的分钟K线
- 财务数据，你可以使用 tushare，akshare或者QMT 任意一种获取

财务指标如何指导量化交易

财务指标如何指导量化交易

Thinking：财务指标如何指导量化交易？

作用是筛选标的和构建因子。

- 选股：从几千只股票中，用财务指标筛出好公司
- 择时/加权：根据财务指标的变化趋势，决定买入/卖出/调整仓位

Thinking：这些财务指标，在实战中，哪些最重要？

- ROE（净资产收益率） —— 选股之王
- 营收/净利润增长率 —— 成长性判断
- 毛利率（grossprofit_margin） —— 护城河指标
- 资产负债率（debt_to_assets） —— 风险控制
- 经营现金流（operating_cashflow） —— 盈利质量
- 总资产周转率（assets_turn） —— 经营效率

指标1：ROE（净资产收益率）

1. ROE（净资产收益率） —— 选股之王

公司用股东的钱赚了多少利润。ROE = 净利润 / 净资产 * 100%

巴菲特最看重的指标。ROE 持续 > 15% 的公司，长期来看大概率是好公司。

```
# 获取贵州茅台全部报告期的财务指标（QMT）
df = extract_indicators('600519.SH')


# 策略：ROE 连续8期（2年）都 > 15% => 优质公司
if df.tail(8)['roe'].dropna().min() > 15:
    print("ROE 连续2年 > 15%，是优质公司")
```

单看一期 ROE 没意义，要看连续性（至少 2 年）

QMT 中 ROE 来自 PershareIndex 表的 du_return_on_equity 字段

指标2：营收/净利润增长率

2. 营收/净利润增长率 —— 成长性判断

和去年同期比，赚钱的速度是快了还是慢了。

股价本质上反映的是预期。持续高增长的公司，市场会给高估值。

增速下滑，即使还在赚钱，股价也可能跌。

```
# QMT 不直接提供同比增长率，需自己用年报数据计算
annual = df[df['end_date'].str.endswith('1231')]  # 只取年报
annual['netprofit_yoy'] = annual['net_profit'].pct_change() *
100

# 策略：净利润增速连续加速（每期比上期增速更高）
annual['growth_accel'] = annual['netprofit_yoy'].diff()
if annual['growth_accel'].tail(3).min() > 0:
    print("净利润增速连续加速，考虑买入")
```

- 净利润同比 > 30% 且 营收同比 > 20% => 高成长
- 净利润增速 > 营收增速 => 盈利质量在提升（成本控制好）
- 净利润增速 < 营收增速 => 增收不增利，警惕

指标3：毛利率

3. 毛利率（grossprofit_margin） -- 护城河指标

毛利率 = (营收 - 成本) / 营收 * 100%

高毛利率意味着定价权（护城河）。毛利率持续下滑，说明竞争加剧或成本失控。

```
# 策略：毛利率近1年下滑超过3个百分点 => 预警
gpm_change = df['grossprofit_margin'].iloc[-1] - df['grossprofit_margin'].iloc[-4]
if gpm_change < -3:
    print(f"警告：毛利率下降 {abs(gpm_change):.1f} 个百分点，竞争力可能减弱")
```

行业参考值：

- 白酒/医药/软件：毛利率 60%~90%（高护城河）
- 制造业/零售：毛利率 15%~30%（竞争激烈）
- 毛利率 < 15%：基本是苦生意

指标4：资产负债率

4. 资产负债率（debt_to_assets） —— 风险控制

资产负债率 = 总负债 / 总资产 * 100%

负债率过高的公司，遇到经济下行容易暴雷。量化交易中用来排雷

```
# 排雷策略：任一条件命中即排除
latest = df.iloc[-1]
if (latest['debt_to_assets'] > 70 or    # 负债率 > 70%
    latest['current_ratio'] < 1.0 or     # 流动比率 < 1
    latest['operating_cashflow'] < 0):   # 经营现金流为负
    print("高风险，加入排雷名单")
```

- 非金融行业：负债率 > 60% 需警惕，> 70% 高危
- 金融/地产行业天然高负债（80%+ 正常），不适用此规则
- 要结合 current_ratio（流动比率）一起看，负债率高但流动比率 > 2 还行

指标5：经营现金流

5. 经营现金流（operating_cashflow）—— 盈利质量

公司主营业务实际收到了多少真金白银。

净利润可以通过会计手段调节，但现金流很难造假。现金流是利润的验钞机。

```
latest = df.iloc[-1]


# 经营现金流/营收 > 15% => 收现能力好
# 经营现金流/净利润 > 1  => 利润含金量高（赚的钱都收回来了）
if latest['ocf_to_revenue'] > 15 and latest['ocf_to_profit'] > 1:
    print("现金流质量好，盈利含金量高")
```

- ocf_to_revenue > 15% => 现金流健康
- ocf_to_profit > 1 => 利润含金量高
- - 连续 2 年经营现金流为负 => 强烈警告，远离

指标6：总资产周转率

6. 总资产周转率（assets_turn）—— 经营效率

公司用 1 元资产能产生多少营收。

周转率体现经营效率。同样的毛利率，周转率高的公司赚钱更快。

杜邦分析：ROE = 净利率 * 周转率 * 杠杆倍数

Thinking：用这个公式如何判断 ROE 的质量？

- 靠净利率高 => 好（有定价权）
- 靠周转率高 => 好（经营效率高）
- 靠杠杆高 => 危险（借钱撑起来的）

以贵州茅台为例：

- 净利率 ~50%，周转率 ~0.35，负债率 ~21%（杠杆 1.27x）
- ROE = 50% * 0.35 * 1.27 = 22.2%

=> 主要靠超高净利率，不靠杠杆，质量非常高

Summary

用法	核心指标	说明
选股入池	ROE, 毛利率, 负债率	从几千只股票中筛出基本面好的候选池
财务排雷	负债率, 流动比率, 现金流	排除可能暴雷的公司
成长股筛选	净利润同比, 营收同比	找增速快且加速的公司
价值股筛选	ROE, BPS, EPS	结合 PE/PB 找被低估的好公司
因子构建	ROE, 毛利率, 周转率	作为多因子模型的 alpha 因子
风险控制	负债率, 速动比率, 现金比率	控制持仓组合的整体风险
行业对比	毛利率, 周转率, ROE	同行业内横向对比，找龙头
趋势跟踪	各指标的环比/同比变化	财务指标恶化 => 减仓/卖出信号

财务指标在量化中的常见用法

Summary（注意事项）

1. 财务指标有滞后性

财务报表是季度发布的（一季报 4月底、半年报 8月底、三季报 10月底、年报 4月底），等你看到数据的时候，已经是 1~4 个月前的事了。所以：

- 不适合做短线交易（日内/周级别）
- 适合做中长线选股（月度/季度调仓）
- 建议每季度财报发布后调仓一次

Summary（注意事项）

2. 不同行业标准不同

同一个指标，在不同行业的正常值差异很大：

指标	白酒/医药	银行/保险	制造业	互联网
ROE	20~35%	10~15%	8~15%	15~30%
毛利率	60~90%	不适用	15~30%	50~80%
负债率	20~40%	85~95%	40~65%	20~50%
周转率	0.3~0.5	不适用	0.8~1.5	0.5~1.0

量化策略中，应该在同行业内对比，而不是跨行业比较。

Summary（注意事项）

3. 单一指标不可靠

任何单一指标都可以被优化或有误导性:

- 高 ROE 可能是靠高杠杆（危险）=> 要结合负债率看
- 高增长可能是一次性收入（卖资产）=> 要结合扣非净利润看
- 低负债可能是因为不投入（不增长）=> 要结合增长率看

永远用多个指标交叉验证 => 多因子模型的核心思想

Summary（注意事项）

4. 财务数据 + 行情数据结合效果更好

最佳实践是财务面 + 技术面结合：

- 用财务指标选出好公司（股票池）
- 用技术指标选择好时机（买卖点）

负债率 > 70%　　=> 直接排除，不参与

经营现金流连续为负 => 直接排除，不参与

ROE > 15%　　+ MACD金叉　　=> 买入

毛利率 > 30%　　+ 股价站上20日均线　=> 加仓

CASE：多因子选股

CASE：多因子选股

TO DO：多因子选股

通过多个财务指标组合筛选，从全市场股票中选出基本面优良的标的。

Thinking：如何筛选好公司？

高盈利 + 高增长 + 高毛利 + 低负债 + 强现金流 = 好公司。

类别	指标	来源/计算
盈利能力	roe, roa, grossprofit_margin, netprofit_margin	PershareIndex / 利润表
偿债能力	debt_to_assets, current_ratio, quick_ratio	资产负债表
营运能力	assets_turn	营收/总资产
现金流	operating_cashflow, ocf_to_revenue, ocf_to_profit	现金流量表
成长能力	netprofit_yoy	年报净利润 pct_change 计算同比

CASE：多因子选股

整体流程：

Step1. 连接 QMT：xtdata.connect()

Step2. 获取股票列表：xtdata.get_stock_list_in_sector('沪深A股')

Step3. 逐只股票处理：

- 调用 download_financial_data2 异步下载 5 张表：Balance、Income、CashFlow、PershareIndex、Capital
- 等待回调完成（最多 60 秒）
- 调用 get_financial_data 获取已下载数据
- 提取各报告期财务指标，取最新一期
- 用年报数据计算净利润同比 netprofit_yoy

CASE：多因子选股

制定5层筛选标准：

- 盈利能力：ROE >= 15%
- 成长能力：净利润同比 >= 10%
- 护城河/定价权：毛利率 >= 30%
- 安全性：资产负债率 <= 60%
- 现金流质量：经营现金流/营收 >= 10%

CASE：多因子选股

```
# 读取候选池
df = pd.read_csv(INPUT_FILE, encoding='utf-8-sig')
# 数值化（CSV 可能读成字符串）
for col in ['roe', 'netprofit_yoy', 'grossprofit_margin', 'debt_to_assets',
'ocf_to_revenue']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# 5 层漏斗（逐层与 mask 相与）
mask = pd.Series(True, index=df.index)
mask &= (df['roe'] >= ROE_MIN)
mask &= (df['netprofit_yoy'] >= NETPROFIT_YOY_MIN)
mask &= (df['grossprofit_margin'] >= GROSSPROFIT_MARGIN_MIN)
mask &= (df['debt_to_assets'] <= DEBT_TO_ASSETS_MAX)
mask &= (df['ocf_to_revenue'] >= OCF_TO_REVENUE_MIN)
selected = df[mask].sort_values('roe', ascending=False)
```

```
# 阈值配置，可在脚本顶部修改
ROE_MIN = 15
NETPROFIT_YOY_MIN = 10
GROSSPROFIT_MARGIN_MIN = 30
DEBT_TO_ASSETS_MAX = 60
OCF_TO_REVENUE_MIN = 10
```

多因子选股-筛选1.py

CASE：多因子选股

多因子选股 - 筛选1（通用版，全行业）

ROE >= 15% | 净利润同比 >= 10% | 毛利率 >= 30%

资产负债率 <= 60% | 经营现金流/营收 >= 10%

读取 5154 只股票

第1层 ROE >= 15%: 剩余 217 只

第2层 净利润同比 >= 10%: 剩余 129 只

第3层 毛利率 >= 30%: 剩余 80 只

第4层 资产负债率 <= 60%: 剩余 74 只

第5层 经营现金流/营收 >= 10%: 剩余 64 只

筛选完成：共 64 只股票达标

已保存：D:\推荐系统\量化交易-2\3-打造数字化办公室\CASE-多因子选股\data\stock_fina_selected_QMT.csv

==========

达标股票（按 ROE 排序）：

多因子选股-筛选1.py

CASE：多因子选股

stock_code	stock_name	end_date	roe	netprofit_yoy	grossprofit_margin	debt_to_assets	ocf_to_revenue
300502.SZ	新易盛	20250930	55.3742	312.26	47.2470	31.9920	28.0974
000752.SZ	ST西发	20250930	38.5122	351.42	42.7305	16.6706	31.1558
300573.SZ	兴齐眼药	20250930	34.7146	40.84	81.3277	23.2356	30.1787
300394.SZ	天孚通信	20250930	33.0869	83.98	51.8674	16.4521	34.3043
300308.SZ	中际旭创	20250930	31.3331	143.33	40.7411	29.4734	21.8151
002847.SZ	盐津铺子	20250930	31.2394	24.78	30.3221	46.6719	13.9830
600809.SH	山西汾酒	20250930	31.0185	17.16	76.0997	27.9976	27.2810
688578.SH	艾力斯	20250930	27.4288	121.97	96.7918	10.4766	46.3523
300476.SZ	胜宏科技	20250930	26.9289	71.96	35.8519	50.4409	16.8828
600519.SH	贵州茅台	20250930	26.3688	15.24	91.2934	12.8088	29.1793
603699.SH	纽威股份	20250930	26.2089	58.77	36.9286	58.0507	29.8617
600329.SH	达仁堂	20250930	25.5407	128.68	74.9830	22.1064	16.0859
605117.SH	德业股份	20250930	24.9846	65.29	38.5467	56.5082	30.0919
688278.SH	特宝生物	20250930	23.9502	49.00	92.5579	20.5213	19.0002
001337.SZ	四川黄金	20250930	23.1395	17.67	64.1119	23.9828	51.0443

多因子选股-筛选1.py

CASE：多因子选股

Thinking：这种筛选方式有什么问题？

没有考虑行业的特点，不同行业的ROE，利润率不同

可以基于行业排名进行打分

行业内排名	得分
前 20%	5 分
20%~40%	4 分
40%~60%	3 分
60%~80%	2 分
后 20%	1 分

5 个指标：ROE、净利润同比、毛利率、资产负债率、经营现金流/营收。

总分范围 5~25，筛选条件：总分 >= SCORE_MIN（默认 18）。

多因子选股-筛选2.py

CASE：多因子选股

多因子选股 - 筛选2（行业版，打分制）

筛选条件：5 项各 1~5 分（按行业内排名），总分 >= 18

读取 5154 只股票

打分制：5 项总分 >= 18（每项 1~5 分，总分 5~25）：剩余 1177 只

筛选完成：共 1177 只股票达标

已保存：D:\推荐系统\量化交易-2\3-打造数字化办公室\CASE-多因子选股\data\stock_fina_selected_QMT_industry.csv

==========

各行业指标分布（全市场）：

==========

	roe			netprofit_yoy			grossprofit_margin			debt_to_assets			ocf_to_revenue		
	mean	median	count	mean	median	count	mean	median	count	mean	median	count	mean	median	count
industry															
SW1交通运输	5.36	5.06	117	-4.36	1.24	117	23.86	21.98	117	44.65	44.26	117	23.99	23.49	117
SW1休闲服务	0.96	3.28	33	-468.84	-16.14	34	29.73	24.79	34	46.14	44.07	34	18.88	14.88	34
SW1传媒	-2.10	2.15	150	-135.78	-30.30	152	28.96	26.90	152	39.91	35.35	152	2.97	2.68	152
SW1公用事业	-5.18	4.92	177	-32.83	-4.50	178	26.66	23.79	178	55.06	55.48	178	20.53	16.36	178
SW1农林牧渔	-4.44	2.89	85	-135.71	-49.78	88	17.70	12.70	88	49.50	49.41	88	8.98	9.45	88

多因子选股-筛选2.py

CASE：多因子选股

SW1化工	-2.59	3.63	380	-9.06	-14.43	381	19.20	17.48	381	43.32	42.88	381	6.57	6.43	381
SW1医药生物	2.11	4.26	351	-49.80	-9.33	351	48.35	48.20	351	33.53	29.33	351	11.77	12.56	351
SW1商业贸易	5.26	2.43	88	17.57	-19.42	88	30.57	26.72	88	53.19	53.05	88	23.68	8.79	88
SW1国防军工	-5.93	1.58	78	-50.37	-35.02	79	26.49	26.69	79	41.27	38.75	79	-14.27	-8.10	79
SW1家用电器	6.39	6.53	67	-54.41	-2.59	67	23.45	21.46	67	47.28	46.83	67	3.20	6.23	67
SW1建筑材料	-3.69	1.35	68	-58.97	-48.39	70	17.13	18.17	70	50.39	44.73	70	2.72	3.88	70
SW1建筑装饰	-8.02	2.24	126	-108.16	-14.72	132	16.25	14.51	131	65.07	66.87	132	-3.68	-5.23	132
SW1房地产	-5.22	0.18	103	-33.86	-24.66	111	20.95	18.39	111	62.22	64.22	111	5.40	4.25	111
SW1有色金属	6.21	5.58	127	-27.05	-4.51	127	16.41	13.52	127	44.99	48.20	127	4.73	3.28	127
SW1机械设备	2.68	3.40	411	3.34	-7.78	413	27.01	25.82	413	44.41	43.42	413	4.80	5.62	413
SW1汽车	3.47	5.55	187	-5.01	4.98	187	18.50	17.24	187	45.79	46.51	187	7.55	7.80	187
SW1电子	2.95	3.98	303	-63.10	-8.83	305	23.68	21.61	305	42.30	41.02	305	8.31	7.31	305
SW1电气设备	3.74	4.16	218	-52.59	-9.36	219	21.20	20.47	219	49.75	50.38	219	4.35	3.71	219
SW1纺织服装	0.43	3.26	81	-157.50	-10.90	82	32.65	30.51	82	40.08	36.78	82	3.81	6.53	82
SW1综合	4.12	2.18	32	-468.97	-19.66	32	31.60	29.97	32	43.74	41.88	32	-1.81	-0.14	32
SW1计算机	-3.46	-0.37	251	-63.20	-32.83	251	36.44	32.94	251	36.68	33.46	251	-10.49	-6.85	251
SW1轻工制造	-8.03	2.59	139	-169.38	-15.21	139	19.52	19.98	139	42.11	39.96	139	2.32	5.30	139

…

多因子选股-筛选2.py

CASE：多因子选股

达标股票（按总分排序）：

==========

stock_code stock_name industry industry_score end_date roe netprofit_yoy grossprofit_margin debt_to_assets ocf_to_revenue roe_industry_score netprofit_yoy_industry_score grossprofit_margin_industry_score debt_to_assets_industry_score ocf_to_revenue_industry_score

688578.SH 艾力斯 SW1医药生物 25.0 20250930 27.4288 121.97 96.7918 10.4766 46.3523 5.0 5.0 5.0 5.0 5.0

002801.SZ 微光股份 SW1电气设备 25.0 20250930 16.6524 92.42 31.0703 15.7062 13.1913 5.0 5.0 5.0 5.0 5.0

300811.SZ 铂科新材 SW1有色金属 25.0 20250930 11.2728 46.24 40.5263 23.1879 24.1252 5.0 5.0 5.0 5.0 5.0

688300.SH 联瑞新材 SW1有色金属 25.0 20250930 14.0097 44.47 41.4071 26.3761 16.0573 5.0 5.0 5.0 5.0 5.0

603893.SH 瑞芯微 SW1电子 25.0 20250930 20.0290 341.01 41.7721 17.0845 26.9724 5.0 5.0 5.0 5.0 5.0

603379.SH 三美股份 SW1化工 25.0 20250930 22.6397 177.08 50.7298 17.4323 35.2429 5.0 5.0 5.0 5.0 5.0

多因子选股-筛选2.py

CASE：多因子选股

688008.SH 澜起科技 SW1电子 25.0 20250930 13.7960 197.18 61.4587 10.8681 39.4479
5.0 5.0 5.0 5.0 5.0
688550.SH 瑞联新材 SW1化工 25.0 20250930 8.9581 87.60 47.6387 10.2879 21.4526 5.0
5.0 5.0 5.0 5.0
300002.SZ 神州泰岳 SW1计算机 25.0 20250930 9.9269 61.25 58.5833 11.0913 15.9447
5.0 5.0 5.0 5.0 5.0
688093.SH 世华科技 SW1电子 25.0 20250930 13.2320 44.56 57.7847 5.1536 33.9092 5.0
5.0 5.0 5.0 5.0
000975.SZ 山金国际 SW1有色金属 25.0 20250930 17.9497 54.95 28.3871 19.1615 24.6227
5.0 5.0 5.0 5.0 5.0
002128.SZ 电投能源 SW1采掘 25.0 20250930 11.4643 16.94 35.5546 25.1754 35.0391 5.0
5.0 5.0 5.0 5.0
003025.SZ 思进智能 SW1机械设备 25.0 20250930 9.5936 86.27 37.6789 19.3860 33.8361
5.0 5.0 5.0 5.0 5.0
600276.SH 恒瑞医药 SW1医药生物 25.0 20250930 10.9521 48.14 86.2195 12.1401 39.2893
5.0 5.0 5.0 5.0 5.0
300641.SZ 正丹股份 SW1化工 25.0 20250930 22.6440 11949.30 43.4736 2.2521 52.2027
5.0 5.0 5.0 5.0 5.0 …

多因子选股-筛选2.py

CASE：多因子选股

Thinking：从图中你能出什么？

ROE 最高行业（Top 5）：

银行(7.28%)、家用电器(6.39%)、有色金属(6.21%)、食品饮料(6.18%)、交通运输(5.36%)，均值约 5%~7%。

ROE 最低行业（Bottom 5）：

轻工制造、建筑装饰、国防军工、房地产、公用事业，均值为负，反映盈利压力较大。

毛利率最高行业：医药生物、食品饮料，均值约 38%~48%，消费与医药具备较高定价能力。

毛利率最低行业：钢铁、建筑装饰、有色金属、建筑材料，均值约 7%~17%，重资产、高成本行业毛利率较低。

资产负债率最高行业：银行、非银金融、建筑装饰、房地产，均值 62%~92%，金融与地产行业杠杆较高。

资产负债率最低行业：医药生物、食品饮料、计算机，均值约 27%~36%，偏轻资产、现金流较好。

CASE：多因子选股

Thinking：为什么不同行业的ROE差异很大？

因为不同行业赚钱的驱动力不同：

- 高销售净利率（茅台模式）：靠品牌溢价（医药、食品）。
- 高资产周转率（沃尔玛模式）：靠薄利多销（零售、贸易）。
- 高权益乘数/杠杆（银行模式）：靠借钱生钱（银行、地产）。

CASE：多因子选股（金融与地产）

分类1、 金融与地产（高杠杆、高 ROE）

代表行业： 银行、非银金融、房地产

数据特征： 资产负债率极高 (62%-92%)，但 ROE 往往不错 (银行 7.28%)。

Thinking：为什么这样？

银行/保险： 它们的负债其实是客户的存款或保费，这是它们的原材料。负债率高是商业模式决定的，不是风险（除非坏账爆发）。

房地产： 典型的资金密集型，靠借贷拿地盖楼。

CASE：多因子选股（金融与地产）

Thinking：选股策略（看什么指标）？

避坑指标： 别看 资产负债率（没意义）、别单纯看 PE（失真）。

核心指标：

- PB（市净率）： 金融股最核心的估值指标。
- 股息率 (Dividend Yield)： 银行股通常作为防御性资产，分红很重要。
- 不良贷款率/拨备覆盖率（银行专用）： 衡量资产质量。
- NAV（净资产价值，地产专用）： 衡量手里地皮的真实价值。

CASE：多因子选股（大消费与医药）

分类2、大消费与医药（高毛利、低负债）

代表行业：食品饮料、医药生物、家用电器 数据特征：毛利率极高 (38%-48%)，负债率低，现金流好。

Thinking：为什么这样？

这是典型的品牌壁垒行业。恒瑞医药（医药）之所以毛利高，是因为有专利保护；贵州茅台（食品）之所以毛利高，是因为品牌溢价。

Thinking：选股策略（看什么指标）？

- GPM（毛利率）稳定性：越高越好，且不能剧烈波动。
- ROE 稳定性：消费股是长跑冠军，我们要找 ROE 连续 3-5 年维持在 15%-20% 以上的公司。
- 销售费用率：注意看为了维持高毛利，公司是否烧了太多的钱在广告上。
- PEG（市盈率相对盈利增长比率）：既然给高估值，必须有高增长匹配。

CASE：多因子选股（周期与制造）

分类3、 周期与制造（低毛利、重资产）

代表行业： 钢铁、有色金属、化工、交通运输、建筑 数据特征： 毛利率惨不忍睹 (7%-17%)，ROE 波动大。

Thinking：为什么这样？

同质化竞争： 你的钢材和他的钢材没区别，只能拼价格。

重资产： 机器设备折旧大，一旦停工亏损严重。

CASE：多因子选股（周期与制造）

Thinking：选股策略（看什么指标）？

避坑指标： 别在 PE 高的时候买（那是周期底部），别在 PE 低的时候买（那是周期顶部）。

核心指标：

- PB-ROE 剪刀差： 在行业复苏初期，寻找低 PB 但 ROE 开始抬头的公司。
- 存货周转率： 谁卖得快，谁就能活下来。
- 大宗商品价格： （作为外部因子）铜价涨，有色股才能涨。
- 现金流 (OCF)： 你的数据里交通运输 OCF 很好，说明虽然毛利低，但收进来的都是真金白银，这类公司适合作为防守仓位。

CASE：多因子选股（TMT与科技）

分类4：TMT与科技（高成长、轻资产）

代表行业： 计算机、电子、传媒、通信 数据特征： 你的数据里显示计算机 ROE 并不高，但我们知道个股分化极大（如满分股瑞芯微）。

Thinking：为什么这样？

赢家通吃： 科技行业要么是 0，要么是 100。大量的研发费用（R&D）在前期会拖累利润，导致 ROE 低，一旦技术突破，爆发力极强。

Thinking：选股策略（看什么指标）？

- 营收增速 (Revenue Growth) > 利润增速： 早期科技股看营收抢占市场。
- 研发支出占比 (R&D/Revenue)： 这是一个衡量“未来潜力”的因子。
- 动量因子 (Momentum)： 科技股往往受情绪和资金驱动，价格趋势很重要。

打卡：多因子选股

扫描市场上的5000+只股票，制定你的选股策略

- 数据可以使用提供的，或者自行下载（使用 tushare, akshare, QMT）
- 你是如何定义好公司的
- 多因子选股的分数制定

Thank You

Using data to solve problems